HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP MÔN HỌC PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

ASSIGNMENT 03

Giảng viên hướng dẫn : TS. Trần Đình Quế

Họ và tên sinh viên : Phạm Ngọc Quynh

Mã sinh viên : B23DCCN704

Nhóm : 06

HÀ NỘI 2026

MỤC LỤC CHI TIẾT BÁO CÁO

A. EXECUTIVE SUMMARY

B. YÊU CẦU CHUNG VÀ PIPELINE HỌC SÂU THUẦN TÚY

- **I. Mô hình toán học của Deep Learning**
- **II. Chu trình 4 giai đoạn của mạng nơ-ron**
- **III. Các thành phần tham số và cấu trúc mạng**
- **IV. So sánh các biến thể Gradient Descent**
- **V. Phân tích toán học các hàm kích hoạt**

C. APPLICATION 1 - DIABETES PREDICTION

- **I. Mô hình và cấu trúc thực nghiệm**
- **II. Kiến trúc mạng neural 5 lớp (22 -> 32 -> 16 -> 8 -> 4 -> 1)**
- **III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)**
  1. Step 1: Đọc và chuẩn bị dữ liệu
  2. Step 2: Chia tập Training Set và Test Set
  3. Step 3: Chuẩn hóa dữ liệu và phòng ngừa Data Leakage
  4. Step 4 & 5: Các hàm kích hoạt ReLU và Sigmoid
  5. Step 6: Khởi tạo trọng số He và bias
  6. Step 7 & 8: Lan truyền tiến (Forward) và bộ nhớ Cache
  7. Step 9: Hàm mất mát Binary Cross-Entropy
  8. Step 10: Lan truyền ngược (Backpropagation) bằng Chain Rule
  9. Step 11 & 12: Vòng lặp huấn luyện Gradient Descent
  10. Step 13 & 14: Dự đoán và đánh giá chi tiết (Confusion Matrix)
- **IV. So sánh với mô hình Machine Learning cũ (SVM RBF)**
- **V. Trực quan hóa và phân tích đồ thị**

D. APPLICATION 2 - HOUSE PRICE PREDICTION

- **I. Mô hình và cấu trúc thực nghiệm**
- **II. Kiến trúc mạng hồi quy 5 lớp (276 -> 32 -> 16 -> 8 -> 4 -> 1)**
- **III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)**
  1. Step 1: Đọc dữ liệu và One-Hot Encoding vị trí
  2. Step 2: Chia tập Train/Test
  3. Step 3: Chuẩn hóa đặc trưng và biến mục tiêu (Target Scaling)
  4. Step 4 & 5: Hàm kích hoạt Linear cho bài toán hồi quy
  5. Step 6 đến 8: Khởi tạo mạng và Lan truyền tiến
  6. Step 9 & 10: Hàm mất mát MSE và Lan truyền ngược hồi quy
  7. Step 11 & 12: Huấn luyện mạng với Gradient Descent
  8. Step 13 & 14: Đổi ngược thang đo và đánh giá chỉ số
- **IV. So sánh với Random Forest và giải thích hiện tượng R^2 âm**
- **V. Trực quan hóa và phân tích đồ thị**

E. APPLICATION 3 - E-COMMERCE CUSTOMER BEHAVIOR

- **I. Mô hình và cấu trúc thực nghiệm**
- **II. Kiến trúc mạng phân loại 5 lớp (5 -> 32 -> 16 -> 8 -> 4 -> 1)**
- **III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)**
  1. Step 1: Chọn lọc đặc trưng dạng bảng và tiền xử lý
  2. Step 2 & 3: Chia Train/Test và chuẩn hóa Z-score
  3. Step 4 đến 12: Lan truyền tiến, hàm mất mát và huấn luyện
  4. Step 13 & 14: Dự đoán ngưỡng 0.5 và ma trận nhầm lẫn
- **IV. So sánh và lưu ý phương pháp luận khi đối chiếu A02**
- **V. Trực quan hóa và phân tích đồ thị**

F. TỔNG KẾT VÀ BÀI HỌC KINH NGHIỆM

I. Bảng tổng hợp đối sánh toàn diện 3 bài toán
II. So sánh giữa Machine Learning kinh điển và Deep Learning
III. Các bài học kỹ thuật cốt lõi
IV. Định hướng nâng cấp mô hình
V. Kết luận

A. EXECUTIVE SUMMARY

Bối cảnh và mục tiêu: Trong môn học Phát triển các hệ thống thông minh, Assignment 02 đã triển khai thành công các giải thuật Machine Learning truyền thống (như Support Vector Machine, Random Forest, Logistic Regression) thông qua các gói công cụ bậc cao của thư viện Scikit-Learn. Tiếp nối hướng nghiên cứu trên 3 tập dữ liệu thực tế đó, Assignment 03 chuyển trọng tâm sang việc tự thiết kế, lập trình và phân tích hoạt động của Mạng nơ-ron học sâu (Deep Neural Network) hoàn toàn từ đầu (from scratch) bằng thư viện đại số tuyến tính cơ bản NumPy, không dựa vào bất kỳ framework học sâu dựng sẵn nào.

Mô hình học sâu được thiết lập thống nhất gồm 5 lớp có tham số (5 trainable layers, bao gồm 4 lớp ẩn và 1 lớp ngõ ra). Quá trình thực nghiệm đối chiếu kết quả của mạng neural tự cài đặt với các mô hình baseline từ Assignment 02 trên cùng tập dữ liệu. Các khái niệm toán học nền tảng như lan truyền tiến (Forward Propagation), hàm kích hoạt (Activation Function), hàm mất mát (Loss Function), lan truyền ngược (Backpropagation) và thuật toán tối ưu Gradient Descent được làm rõ chi tiết qua từng bước tính toán ma trận.

Bảng tổng hợp đối chiếu kết quả thực nghiệm giữa Baseline A02 và Deep Learning A03:

Bài toán (Ứng dụng)	Baseline A02 (ML kinh điển)	Deep Learning A03 (NumPy)	Kết luận & Nhận xét chính
Application 1: Diabetes Prediction (Dự đoán tiểu đường)	SVM (RBF Kernel): • Accuracy: 0.7449 • Precision: 0.7287 • Recall: 0.8343 • F1-Score: 0.7779 • ROC-AUC: 0.8048	5-layer NN (NumPy): • Accuracy: 0.7144 • Precision: 0.7074 • Recall: 0.7866 • F1-Score: 0.7449 (Train loss: 0.6546 → 0.5625)	Cùng biến mục tiêu Diabetes_binary. Mô hình SVM cũ tốt hơn mạng neural ở cả 4 chỉ số do tối ưu được biên phân tách phi tuyến cực đại.
Application 2: House Price Prediction (Dự đoán giá nhà)	Random Forest Regressor: • MAE: 6,303.08 • RMSE: 32,544.55 • R^2: 0.1818 (Có tiền xử lý Robust & Log)	5-layer NN Regression: • MAE: 31,585,039.76 • RMSE: 1,502,158,095.88 • R^2: -2.35175×10^9 (Loss chuẩn hóa: 1.011 → 0.884)	Cùng target giá tiền. Mạng neural cơ bản dùng MSE bị ảnh hưởng nặng nề bởi các điểm ngoại lai (outliers), dẫn đến R^2 âm rất lớn trên tập test.
Application 3: E-Commerce Behavior (Hành vi khách hàng)	Logistic Regression: • Target: Department Name (6 lớp) • Accuracy: 0.8296 • F1-Macro: 0.6339 (Dùng 8,000 từ TF-IDF)	5-layer NN (NumPy): • Target: Recommended IND (2 lớp) • Accuracy: 0.9314 • Precision: 0.9635 • Recall: 0.9530 • F1-Score: 0.9582	Target khác nhau (phân loại 6 lớp văn bản vs phân loại nhị phân dữ liệu bảng), do đó không được kết luận mô hình nào tốt hơn chỉ dựa trên số đo metric.

Đánh giá tổng quan từ kết quả thực nghiệm: Kết quả thu được từ 3 ứng dụng mang lại kết luận khoa học quan trọng: Việc gia tăng số lớp nơ-ron không mặc định đem lại kết quả tốt hơn. Một mạng nơ-ron sâu chỉ phát huy được ưu thế khi đi kèm với việc xử lý ngoại lai cẩn thận, chuẩn hóa dữ liệu đúng phương pháp, và sử dụng các cơ chế tối ưu hóa phù hợp với đặc thù của dữ liệu.

B. YÊU CẦU CHUNG VÀ PIPELINE HỌC SÂU THUẦN TÚY

I. Mô hình toán học của Deep Learning

Khái niệm cốt lõi: Theo định nghĩa trong slide bài giảng, Deep Learning về bản chất là phép hợp thành (composition) của nhiều hàm toán học liên tiếp: ŷ = f3(f2(f1(X))). Thay vì chỉ dùng một phép biến đổi tuyến tính đơn giản như trong Linear Regression hay Logistic Regression, mạng nơ-ron sâu xếp chồng nhiều lớp biến đổi liên tiếp lên nhau.

Trong Assignment 03, cấu trúc này được mở rộng thành 5 lớp có tham số học được (gồm 4 lớp ẩn và 1 lớp ngõ ra):

> ŷ = f5(f4(f3(f2(f1(X)))))

Trong đó, mỗi lớp fk thực hiện hai thao tác cơ bản: (1) Biến đổi affine Z_k = H_{k-1} @ W_k + b_k, và (2) Áp dụng hàm kích hoạt phi tuyến H_k = activation(Z_k). Dữ liệu sau khi đi qua mỗi lớp sẽ được chuyển hóa thành một biểu diễn đặc trưng mới (representation learning). Các biểu diễn ở các lớp sâu hơn sẽ phản ánh các mối tương quan phi tuyến phức tạp hơn giữa các đặc trưng ban đầu.

II. Chu trình 4 giai đoạn của mạng nơ-ron

Quá trình học tập của mạng nơ-ron vận hành tuần hoàn theo một chu trình khép kín gồm 4 bước liên tiếp:

- **1. Forward Propagation (Lan truyền tiến):** Dữ liệu đầu vào X được nhân với ma trận trọng số W và cộng vector độ dời b qua từng lớp, sau đó đi qua hàm kích hoạt phi tuyến để tạo ra giá trị dự đoán ŷ ở ngõ ra. Toàn bộ các giá trị trung gian (pre-activations Z và activations H) được lưu trữ vào bộ nhớ đệm cache.
- **2. Loss Computation (Tính toán hàm mất mát):** So sánh đầu ra dự đoán ŷ với nhãn thực tế y bằng hàm mất mát (Binary Cross-Entropy cho bài toán phân loại nhị phân và Mean Squared Error cho bài toán hồi quy). Hàm mất mát phản ánh mức độ sai lệch định lượng giữa dự đoán và thực tế.
- **3. Backpropagation (Lan truyền ngược):** Sử dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm của hàm mất mát theo từng ma trận trọng số (dW) và vector độ dời (db), đi ngược chiều từ lớp ngõ ra trở về lớp đầu tiên. Bộ nhớ đệm cache được truy xuất để lấy lại các giá trị kích hoạt mà không phải tính lại forward pass.
- **4. Parameter Update (Cập nhật tham số):** Điều chỉnh toàn bộ tham số của mạng theo hướng ngược chiều gradient nhằm giảm giá trị mất mát: W = W - η * dW và b = b - η * db, trong đó η là tốc độ học (learning rate).

III. Các thành phần tham số và cấu trúc mạng

- **Trọng số liên kết (Weights - W):** Là ma trận tham số có kích thước (fan_in, fan_out), thể hiện mức độ đóng góp của từng đặc trưng lớp trước vào nơ-ron của lớp sau.
- **Độ dời (Bias - b):** Là vector kích thước (1, fan_out), giúp dịch chuyển ngưỡng kích hoạt của nơ-ron, giúp đường ranh giới không bị cố định phải đi qua gốc tọa độ trong không gian đặc trưng.
- **Depth vs Width:** Depth (độ sâu) là số lớp biến đổi nối tiếp nhau trong mạng; Width (độ rộng) là số lượng nơ-ron trong từng lớp. Slide bài giảng nhấn mạnh rằng bản chất của học sâu nằm ở việc tạo ra các biểu diễn trừu tượng phân cấp theo chiều sâu, chứ không phải chỉ là việc đặt thật nhiều nơ-ron vào một lớp duy nhất.

- **Hàm kích hoạt (Activation):** Các lớp ẩn bắt buộc phải sử dụng hàm kích hoạt phi tuyến (ReLU) để phá vỡ tính tuyến tính. Nếu không có hàm phi tuyến, tích của 5 ma trận liên tiếp X @ W1 @ W2 @ W3 @ W4 @ W5 chỉ tương đương với một phép nhân ma trận đơn lẻ X @ W_total, làm mất hoàn toàn ý nghĩa của mạng sâu. Lớp ngõ ra dùng Sigmoid để tính xác suất cho bài toán phân loại nhị phân, hoặc dùng Linear cho bài toán hồi quy.

IV. So sánh các biến thể Gradient Descent

Trong lý thuyết tối ưu hóa học sâu, có 3 biến thể cơ bản của thuật toán Gradient Descent. Bảng so sánh dưới đây làm rõ ưu và nhược điểm của từng phương pháp:

Phương pháp tối ưu	Quy mô dữ liệu mỗi bước cập nhật	Ưu điểm nổi bật	Nhược điểm chính
Full-Batch GD (Áp dụng trong A03)	Toàn bộ tập Train (N quan sát)	Gradient cực kỳ ổn định, hội tụ mượt mà, tính toán vector hóa tối ưu bằng NumPy.	Đòi hỏi bộ nhớ RAM lớn, dễ kẹt ở điểm yên ngựa (saddle points), cập nhật chậm với dữ liệu khổng lồ.
Stochastic GD (SGD)	Chỉ 1 mẫu duy nhất mỗi bước	Cập nhật tham số siêu nhanh, có thể nhảy thoát khỏi các cực tiểu cục bộ kém.	Đường đi gradient bị dao động dữ dội, khó hội tụ chính xác ở điểm cực tiểu tối ưu.
Mini-Batch GD	Một lô nhỏ mẫu (32, 64, 128, 256)	Cân bằng hoàn hảo giữa tính ổn định của Full-Batch và tốc độ của SGD; chuẩn mực trong học sâu hiện đại.	Cần tinh chỉnh thêm siêu tham số kích thước lô (batch size).

V. Phân tích toán học các hàm kích hoạt

Hàm kích hoạt đóng vai trò quyết định tính phi tuyến của mạng. Bảng dưới đây phân tích đặc tính toán học của các hàm kích hoạt phổ biến:

Hàm kích hoạt	Công thức toán học f(x)	Đạo hàm f'(x)	Đặc điểm và ứng dụng
ReLU (Tầng ẩn A03)	f(x) = max(0, x)	f'(x) = 1 nếu x > 0; f'(x) = 0 nếu x ≤ 0	Tính toán cực nhanh, tránh triệt tiêu gradient khi x > 0; tiêu chuẩn cho các tầng ẩn.
Sigmoid (Ngõ ra Phân loại)	f(z) = 1 / (1 + e^(-z))	f'(z) = f(z) * (1 - f(z))	Nén giá trị về khoảng (0, 1), diễn giải xác suất; dùng ở ngõ ra phân loại nhị phân.
Linear (Ngõ ra Hồi quy)	f(z) = z	f'(z) = 1	Giữ nguyên dải giá trị thực không bị chặn; tiêu chuẩn cho bài toán hồi quy liên tục.
Tanh	f(z) = (e^z - e^(-z)) / (e^z + e^(-z))	f'(z) = 1 - f(z)²	Đối xứng quanh gốc 0 (zero-centered), nhưng vẫn bị bão hòa gradient ở hai đầu cực.

C. APPLICATION 1 - DIABETES PREDICTION

I. Mô hình và cấu trúc thực nghiệm

1. Mô tả bài toán: Notebook a03(1).ipynb thực hiện dự đoán nguy cơ mắc bệnh tiểu đường dựa trên tập dữ liệu chuẩn hóa diabetes_clean.csv (bắt nguồn từ bộ dữ liệu giám sát sức khỏe BRFSS của CDC Hoa Kỳ).

- **Biến mục tiêu:** Biến mục tiêu là cột Diabetes_binary ∈ {0, 1}, thuộc bài toán Phân loại nhị phân (0: không mắc bệnh, 1: có nguy cơ hoặc mắc tiểu đường).
- **Kích thước dữ liệu:** Dữ liệu gồm 70,841 bản ghi đã được cân bằng tỷ lệ nhãn 50-50. Sau khi đọc và điền giá trị thiếu bằng 0, kích thước ma trận là X = (70,841, 22) và vector nhãn y = (70,841, 1).

Bảng danh mục chi tiết 22 đặc trưng lâm sàng và thói quen sinh hoạt trong tập dữ liệu:

Tên đặc trưng	Ý nghĩa lâm sàng	Tên đặc trưng	Ý nghĩa lâm sàng
HighBP	Huyết áp cao (0/1)	**Smoker**	Hút thuốc ít nhất 100 điếu (0/1)
HighChol	Cholesterol cao (0/1)	**Stroke**	Từng bị đột quỵ (0/1)
CholCheck	Kiểm tra cholesterol 5 năm (0/1)	**HeartDiseaseorAttack**	Bệnh động mạch vành / nhồi máu (0/1)
BMI	Chỉ số khối cơ thể liên tục	**PhysActivity**	Vận động thể chất 30 ngày qua (0/1)
Fruits	Ăn trái cây ít nhất 1 lần/ngày (0/1)	**Veggies**	Ăn rau xanh ít nhất 1 lần/ngày (0/1)
HvyAlcoholConsump	Uống nhiều rượu bia (0/1)	**AnyHealthcare**	Có bảo hiểm y tế bất kỳ (0/1)
NoDocbcCost	Không đi khám do chi phí (0/1)	**GenHlth**	Tự đánh giá sức khỏe (1 tốt - 5 kém)
MentHlth	Số ngày sức khỏe tâm thần kém (0-30)	**PhysHlth**	Số ngày sức khỏe thể chất kém (0-30)
DiffWalk	Khó khăn nghiêm trọng khi đi lại (0/1)	**Sex**	Giới tính sinh học (0 nữ, 1 nam)
Age	Thang độ tuổi 13 cấp độ	**Education**	Trình độ học vấn 6 cấp độ
Income	Thang thu nhập hộ gia đình 8 cấp độ	**Diabetes_binary**	Biến mục tiêu nhị phân (Target 0/1)

2. Mô hình Machine Learning cũ đối chiếu: Trong Assignment 02, mô hình Support Vector Machine với nhân RBF (SVM RBF) được chọn làm mô hình chuẩn. SVM tìm siêu phẳng có khoảng cách biên phân cách lớn nhất giữa hai lớp, và nhân RBF giúp ánh xạ dữ liệu sang không gian phi tuyến. Kết quả kiểm thử baseline cũ đạt: Accuracy = 0.7449, Precision = 0.7287, Recall = 0.8343, F1 = 0.7779, ROC-AUC = 0.8048.

II. Kiến trúc mạng neural 5 lớp (22 -> 32 -> 16 -> 8 -> 4 -> 1)

Mô hình A03 xây dựng mạng nơ-ron gồm 5 lớp có tham số với kiến trúc hình phễu thu hẹp dần: 22 → 32 → 16 → 8 → 4 → 1. Bốn lớp ẩn dùng hàm kích hoạt ReLU, lớp ngõ ra dùng hàm kích hoạt Sigmoid.

Lớp (Stage)	Kích thước biến đổi	Kích thước đầu ra	Hàm kích hoạt	Ý nghĩa / Vai trò
Input	Không có	N × 22	Không	22 đặc trưng ban đầu của bệnh nhân
Layer 1	W1: 22×32, b1: 1×32	N × 32	ReLU	Học biểu diễn đặc trưng tầng thứ nhất (736 tham số)

Layer 2	W2: 32×16, b2: 1×16	N × 16	ReLU	Học biểu diễn mới từ biểu diễn của Layer 1 (528 tham số)
Layer 3	W3: 16×8, b3: 1×8	N × 8	ReLU	Nén và gom cụm các mẫu đặc trưng (136 tham số)
Layer 4	W4: 8×4, b4: 1×4	N × 4	ReLU	Biểu diễn sâu 4 chiều trước khi ra quyết định (36 tham số)
Layer 5 (Output)	W5: 4×1, b5: 1×1	N × 1	Sigmoid	Tính điểm xác suất thuộc lớp tiểu đường (5 tham số)

Tổng số tham số học được của toàn mạng là: (22×32+32) + (32×16+16) + (16×8+8) + (8×4+4) + (4×1+1) = 736 + 528 + 136 + 36 + 5 = 1,441 tham số.

III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)

1. Step 1 - Đọc và chuẩn bị dữ liệu: Đọc tệp CSV vào DataFrame bằng lệnh: df = pd.read_csv(Path('../data/diabetes_clean.csv')). Sau đó áp dụng hàm điền giá trị thiếu: df = df.fillna(0). Thao tác này thay thế tất cả giá trị NaN bằng 0. Tiếp theo, tách đặc trưng đầu vào X và biến mục tiêu y bằng .to_numpy(dtype=float):

```
target_col = 'Diabetes_binary'
X = df.drop(columns=[target_col]).to_numpy(dtype=float)
y = df[[target_col]].to_numpy(dtype=float)
```

Dữ liệu được chuyển thành mảng NumPy 2 chiều để phục vụ các phép nhân ma trận vector hóa.

2. Step 2 - Chia tập Training Set và Test Set: Dùng np.random.seed(42) để cố định bộ sinh số ngẫu nhiên, giúp kết quả có thể tái lập được giữa các lần chạy khác nhau. Tạo hoán vị ngẫu nhiên chỉ số index bằng indices = np.random.permutation(len(X)). Cắt 80% đầu tiên làm tập huấn luyện (train_idx = indices[:split], 56,672 mẫu) và 20% còn lại làm tập kiểm thử (test_idx = indices[split:], 14,169 mẫu).

3. Step 3 - Chuẩn hóa dữ liệu và phòng ngừa Data Leakage: Áp dụng phương pháp chuẩn hóa Z-score: x' = (x - μ) / σ. mean và std được tính toán riêng biệt cho từng cột trên tập train:

```
mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-8
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

Lưu ý quan trọng: Tập X_test bắt buộc phải dùng đúng mean và std đã tính từ X_train, không được tự tính lại. Nếu tính mean/std trên toàn bộ tập dữ liệu trước khi chia, hoặc tự tính riêng trên tập test, thông tin phân phối của tập test sẽ bị rò rỉ vào quá trình tiền xử lý (Data Leakage), dẫn đến kết quả đánh giá không khách quan.

4. Step 4 & 5 - Các hàm kích hoạt ReLU và Sigmoid: Hàm ReLU được định nghĩa là: relu(x) = np.maximum(0, x). Đạo hàm của ReLU là: relu_derivative(x) = (x > 0).astype(float). ReLU giúp đạo hàm không bị triệt tiêu khi giá trị dương, giải quyết bài toán vanishing gradient.

Hàm Sigmoid được định nghĩa cho lớp ngõ ra: sigmoid(z) = 1.0 / (1.0 + np.exp(-x)). Trong mã nguồn, giá trị logit z được giới hạn bằng lệnh x = np.clip(z, -50, 50) trước khi tính hàm exp để chống tràn số học (overflow/underflow).

5. Step 6 - Khởi tạo trọng số He và bias: Trọng số của 5 lớp được khởi tạo theo phương pháp Kaiming He (He initialization) kết hợp với căn bậc hai của fan_in:

```
input_dim = X.shape[1] # 22
W1 = np.random.randn(input_dim, 32) * np.sqrt(2.0 / input_dim)
b1 = np.zeros((1, 32))
W2 = np.random.randn(32, 16) * np.sqrt(2.0 / 32)
b2 = np.zeros((1, 16))
# ... tương tự cho W3, W4, W5 và b3, b4, b5
```

Hệ số √(2/fan_in) giúp giữ cho phương sai của tín hiệu đầu ra ở mỗi lớp không bị nổ lớn hoặc tiêu biến về 0 khi đi qua các lớp kích hoạt ReLU. Các vector bias ban đầu được khởi tạo bằng 0.

6. Step 7 & 8 - Lan truyền tiến (Forward) và bộ nhớ Cache: Hàm forward(X) tính toán tuần tự qua 5 lớp. Điểm quan trọng nhất ở bước này là lưu toàn bộ giá trị đầu vào X, các pre-activations (z1..z5), các activations (h1..h4) và kết quả dự đoán y_hat vào từ điển cache:

```
z1 = X @ W1 + b1;  h1 = relu(z1)
z2 = h1 @ W2 + b2; h2 = relu(z2)
z3 = h2 @ W3 + b3; h3 = relu(z3)
z4 = h3 @ W4 + b4; h4 = relu(z4)
z5 = h4 @ W5 + b5; y_hat = sigmoid(z5)
cache = {'X': X, 'z1': z1, 'h1': h1, 'z2': z2, 'h2': h2, 'z3': z3, 'h3': h3, 'z4': z4, 'h4': h4, 'z5': z5, 'y_hat': y_hat}
```

Tại sao bắt buộc phải lưu cache? Trong lan truyền ngược (Backpropagation), để tính đạo hàm dW5 cần sử dụng h4 (dW5 = h4.T @ dz5), để tính dz4 cần sử dụng z4 (đạo hàm ReLU'(z4)). Nếu không lưu lại các giá trị trung gian này, việc phải tính toán lại toàn bộ forward pass từ đầu sẽ gây lãng phí tài nguyên nghiêm trọng.

7. Step 9 - Hàm mất mát Binary Cross-Entropy: Đo lường sai số bằng hàm mất mát Binary Cross-Entropy (BCE):

```
def binary_cross_entropy(y, y_hat):
    eps = 1e-8
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
```

Giá trị y_hat được kẹp trong khoảng [1e-8, 1 - 1e-8] để ngăn chặn phép tính log(0) sinh ra giá trị không xác định (NaN). Loss phạt rất nặng khi mô hình dự đoán xác suất thấp cho lớp thực tế.

8. Step 10 - Lan truyền ngược (Backpropagation) bằng Chain Rule: Lan truyền ngược áp dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm ngược từ lớp 5 về lớp 1. Khi kết hợp hàm kích hoạt Sigmoid và hàm mất mát BCE, đạo hàm của loss theo logit z5 rút gọn rất đẹp: dz5 = (y_hat - y) / n. Sau đó, gradient được truyền ngược tuần tự:

```
# Lớp 5 (Ngõ ra)
dz5 = (y_hat - y) / n
dW5 = h4.T @ dz5
db5 = np.sum(dz5, axis=0, keepdims=True)

# Lớp 4
dh4 = dz5 @ W5.T
dz4 = dh4 * relu_derivative(z4)
dW4 = h3.T @ dz4
db4 = np.sum(dz4, axis=0, keepdims=True)
# ... tiếp tục truyền ngược về Lớp 3, Lớp 2 và Lớp 1
```

9. Step 11 & 12 - Vòng lặp huấn luyện Gradient Descent: Vòng lặp huấn luyện lặp lại 1000 epoch với tốc độ học learning_rate = 0.01. Trong mỗi epoch: Forward → Tính Loss → Backward → Cập nhật W và b. Giá trị loss giảm dần từ 0.6546 ở epoch 100 xuống 0.5625 ở epoch 1000:

```
Epoch 100/1000, Loss: 0.6546
Epoch 200/1000, Loss: 0.6348
Epoch 300/1000, Loss: 0.6197
Epoch 400/1000, Loss: 0.6066
Epoch 500/1000, Loss: 0.5953
Epoch 600/1000, Loss: 0.5861
Epoch 700/1000, Loss: 0.5784
Epoch 800/1000, Loss: 0.5720
Epoch 900/1000, Loss: 0.5668
Epoch 1000/1000, Loss: 0.5625
```

10. Step 13 - Dự đoán trên tập kiểm thử (Prediction): Đưa tập X_test qua hàm forward để nhận xác suất dự đoán y_prob. Sau đó dùng ngưỡng quyết định 0.5 để chuyển thành nhãn nhị phân: y_pred = (y_prob >= 0.5).astype(int).

11. Step 14 - Đánh giá chi tiết (Metrics & Confusion Matrix): Trên tập test gồm 14,169 bệnh nhân, mô hình đạt được các chỉ số:

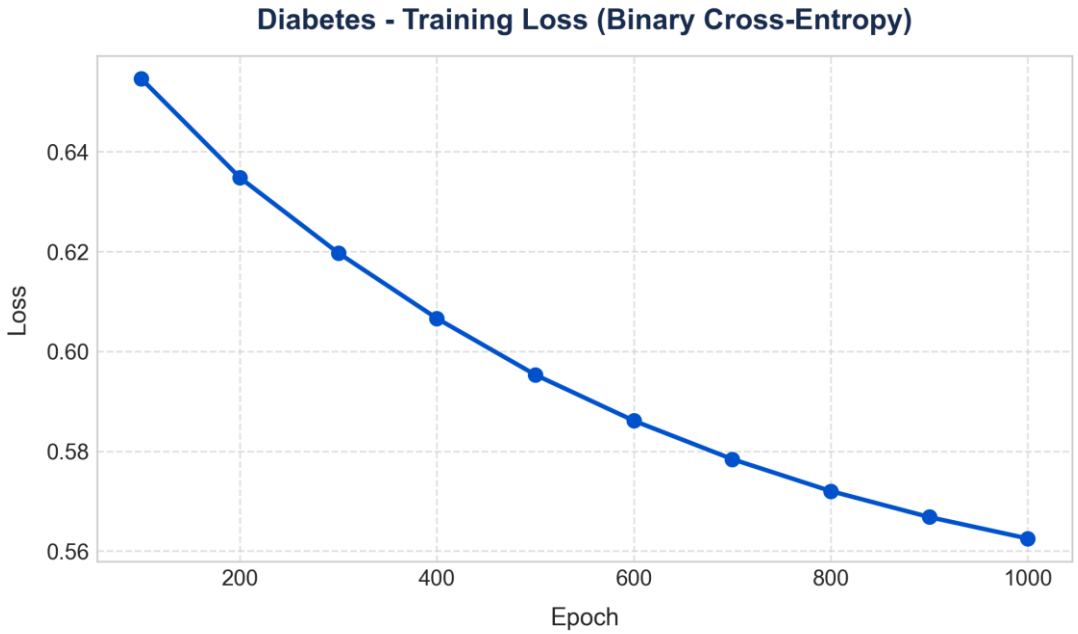
- **Độ chính xác (Accuracy):** Accuracy = 0.714376 (71.44%): Tỷ lệ dự đoán đúng trên toàn bộ tập test.
- **Độ chuẩn xác (Precision):** Precision = 0.707375 (70.74%): Trong các ca mô hình dự đoán mắc bệnh, có 70.74% là đúng thực tế.
- **Độ bao phủ (Recall):** Recall = 0.786580 (78.66%): Trong các ca thực tế mắc bệnh, mô hình phát hiện được 78.66%.
- **F1-Score:** F1-Score = 0.744878 (74.49%): Trung bình điều hòa giữa Precision và Recall.
- **Ma trận nhầm lẫn:** TN = 4,214 (Âm tính đúng); FP = 2,444 (Dương tính giả); FN = 1,603 (Âm tính giả - bỏ sót); TP = 5,908 (Dương tính đúng).

IV. So sánh với mô hình Machine Learning cũ (SVM RBF)

Mô hình	Accuracy	Precision	Recall	F1-Score
SVM (RBF Kernel) - A02	0.7449	0.7287	0.8343	0.7779
5-layer NN - A03	0.7144	0.7074	0.7866	0.7449

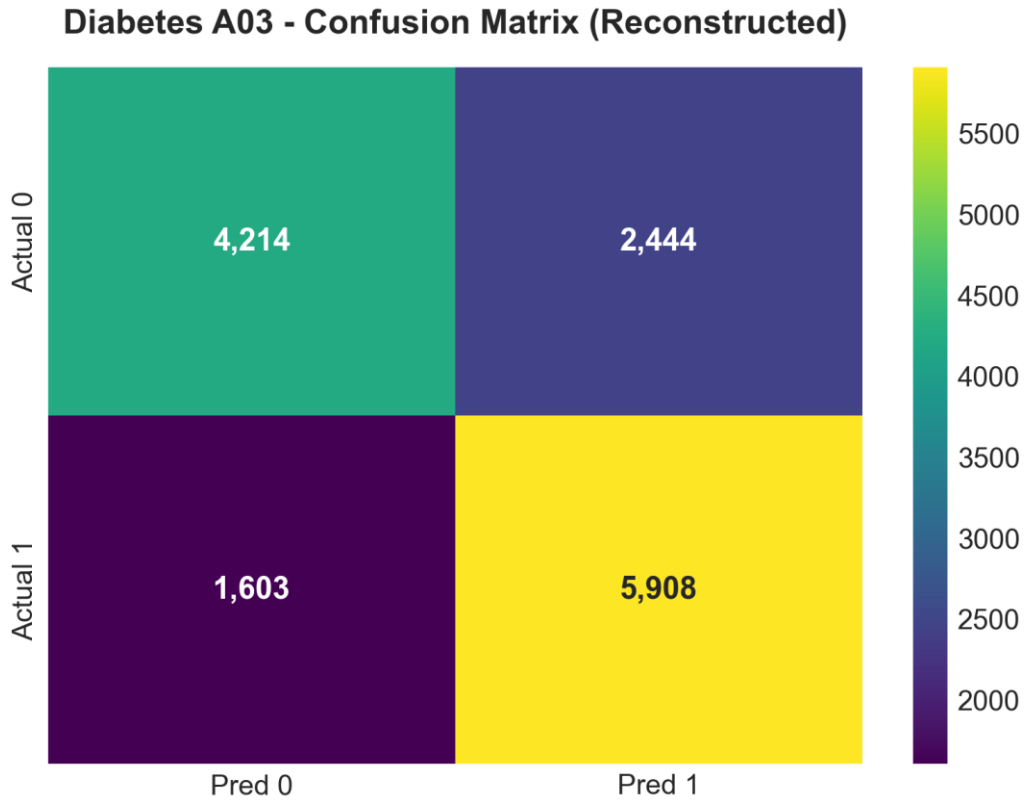
Nhận xét kết quả: Mô hình SVM cũ tốt hơn mạng neural ở cả 4 chỉ số. Đặc biệt trong chẩn đoán y tế, Recall là chỉ số quan trọng nhất (vì một ca bệnh bị bỏ sót FN gây nguy hiểm hơn việc báo động nhầm FP). SVM đạt Recall 0.8343 so với 0.7866 của mạng nơ-ron. Nguyên nhân là do SVM với RBF kernel tối ưu hóa trực tiếp khoảng cách biên cực đại giữa hai lớp trong không gian đặc trưng; trong khi mạng nơ-ron A03 là mô hình cài đặt cơ bản từ đầu, dùng Full-batch GD với learning rate cố định, chưa có cơ chế điều hòa (regularization) hay tối ưu siêu tham số.

V. Trực quan hóa và phân tích đồ thị

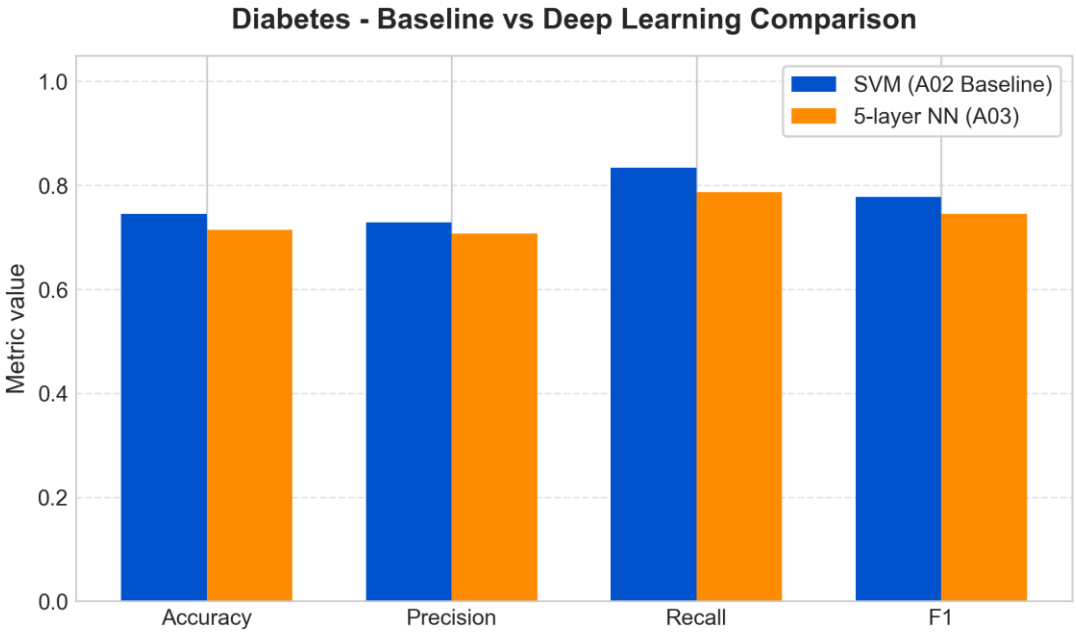


Hình 1: Đồ thị hàm mất mát Binary Cross-Entropy theo epoch của bài toán Diabetes.

Đồ thị Hình 1 cho thấy đường loss giảm đều từ 0.6546 xuống 0.5625. Tốc độ giảm chậm dần ở các epoch sau, chứng tỏ mô hình đang hội tụ ổn định.



Hình 2: Ma trận nhầm lẫn của mô hình mạng neural 5 lớp trên tập test.



Hình 3: Biểu đồ cột so sánh trực tiếp 4 chỉ số giữa SVM cũ và Mạng neural 5 lớp.

D. APPLICATION 2 - HOUSE PRICE PREDICTION

I. Mô hình và cấu trúc thực nghiệm

1. Mô tả bài toán: Notebook a03(2).ipynb thực hiện dự đoán giá bất động sản từ tệp house_price_clean.csv. Biến mục tiêu là price_million_vnd (giá nhà tính bằng triệu VNĐ). Đây là bài toán hồi quy (regression) vì biến mục tiêu là một giá trị số thực liên tục. Cột vị trí (location) được mã hóa One-Hot (drop_first=True). Sau khi mã hóa và điền khuyết bằng 0, tập dữ liệu có kích thước X = (64,447, 276) và y = (64,447, 1).

2. Mô hình Baseline cũ (Random Forest): Trong Assignment 02, thuật toán Random Forest Regressor được chọn làm mô hình chuẩn với kết quả: MAE = 6,303.08; RMSE = 32,544.55; R^2 = 0.1818. Pipeline cũ đã áp dụng kỹ thuật tiền xử lý phức tạp gồm RobustScaler, missing indicators và biến đổi log1p cho biến mục tiêu.

II. Kiến trúc mạng hồi quy 5 lớp (276 -> 32 -> 16 -> 8 -> 4 -> 1)

Mô hình A03 sử dụng mạng 5 lớp: 276 → 32 → 16 → 8 → 4 → 1. Bốn lớp ẩn dùng hàm kích hoạt ReLU. Riêng lớp ngõ ra dùng hàm kích hoạt Linear (đồng nhất) thay vì Sigmoid, vì giá nhà là đại lượng liên tục không bị chặn trên trong khoảng [0, 1].

Lớp (Stage)	Kích thước biến đổi	Kích thước đầu ra	Hàm kích hoạt	Ý nghĩa / Vai trò
Input	Không có	N × 276	Không	276 đặc trưng sau One-Hot vị trí
Layer 1	W1: 276×32, b1: 1×32	N × 32	ReLU	Học biểu diễn tầng 1 từ 276 đặc trưng (8,864 tham số)
Layer 2	W2: 32×16, b2: 1×16	N × 16	ReLU	Học biểu diễn mới từ tầng 1 (528 tham số)
Layer 3	W3: 16×8, b3: 1×8	N × 8	ReLU	Nén và kết hợp các đặc trưng bất động sản (136 tham số)
Layer 4	W4: 8×4, b4: 1×4	N × 4	ReLU	Biểu diễn sâu 4 chiều (36 tham số)
Layer 5 (Output)	W5: 4×1, b5: 1×1	N × 1	Linear	Dự đoán giá trị hồi quy đã chuẩn hóa (5 tham số)

Tổng số tham số của mạng hồi quy là: (276×32+32) + (32×16+16) + (16×8+8) + (8×4+4) + (4×1+1) = 8,864 + 528 + 136 + 36 + 5 = 9,569 tham số.

III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)

1. Step 1 - Tiền xử lý dữ liệu: Đọc dữ liệu và mã hóa One-Hot cho cột địa điểm: df = pd.get_dummies(df, columns=['location'], drop_first=True). Tham số drop_first=True bỏ đi cột đầu tiên làm mốc để tránh dư thừa đa cộng tuyến (Dummy Variable Trap). Sau đó điền các ô thiếu còn lại bằng 0: df = df.fillna(0). Tách X và y sang định dạng NumPy ndarray.

2. Step 2 - Chia tập Train/Test: Chia 80% train (51,557 mẫu) và 20% test (12,890 mẫu) bằng cách tạo hoán vị ngẫu nhiên với np.random.seed(42).

3. Step 3 - Chuẩn hóa đặc trưng và biến mục tiêu (Target Scaling): Chuẩn hóa đặc trưng X bằng mean và std của X_train. Điểm đặc biệt ở bài toán hồi quy giá nhà là việc chuẩn hóa cả biến mục tiêu y là bắt buộc:

```
y_mean = y_train.mean()
y_std = y_train.std() + 1e-8
y_train_scaled = (y_train - y_mean) / y_std
y_test_scaled = (y_test - y_mean) / y_std
```

Tại sao cần chuẩn hóa biến mục tiêu trong hồi quy? Giá bất động sản có giá trị thực rất lớn (từ hàng trăm triệu đến hàng trăm tỷ đồng). Nếu giữ nguyên giá trị gốc để tính hàm mất mát bình phương MSE, sai số sẽ lên tới 10^16, khiến đạo hàm bùng nổ (gradient explosion) và làm mô hình tràn số ngay từ vòng lặp đầu tiên. Mạng sẽ học trên thang đo y_train_scaled, sau đó khi dự đoán sẽ đổi ngược lại về đơn vị ban đầu.

4. Step 4 & 5 - Hàm kích hoạt Linear ngõ ra: Lớp ngõ ra dùng hàm kích hoạt đồng nhất (Linear): linear(z) = z, đạo hàm là linear_derivative(a) = 1. Đạo hàm bằng 1 được tích hợp sẵn vào gradient của MSE nên không cần gọi riêng trong backward pass.

5. Step 6 đến 8 - Khởi tạo mạng và Lan truyền tiến: Khởi tạo trọng số W1..W5 theo phân phối He với căn bậc hai √(2/fan_in) và bias b1..b5 bằng 0. Quá trình lan truyền tiến forward() tương tự như Application 1, chỉ khác ở lớp cuối cùng y_hat = linear(z5). Các giá trị được lưu vào cache.

6. Step 9 & 10 - Hàm mất mát MSE và Lan truyền ngược: Hàm mất mát là Mean Squared Error (MSE): L = (1/N) ∑ (y_scaled - ŷ_scaled)². Đạo hàm của MSE theo giá trị dự đoán ŷ là:

```
dz5 = 2.0 * (y_hat - y_train_scaled) / n
```

Số 2 xuất phát từ đạo hàm của (ŷ - y)². Từ dz5, gradient tiếp tục được truyền ngược về các lớp ẩn qua công thức dW5 = h4.T @ dz5, dh4 = dz5 @ W5.T, dz4 = dh4 * relu_derivative(z4), v.v.

7. Step 11 & 12 - Huấn luyện với Gradient Descent: Huấn luyện 1000 epoch với learning rate 0.01. Loss MSE trên thang đo chuẩn hóa giảm từ 1.0111 ở epoch 100 xuống 0.8836 ở epoch 1000:

```
Epoch 100/1000, Scaled MSE Loss: 1.0111
Epoch 200/1000, Scaled MSE Loss: 0.9950
Epoch 300/1000, Scaled MSE Loss: 0.9870
Epoch 400/1000, Scaled MSE Loss: 0.9760
Epoch 500/1000, Scaled MSE Loss: 0.9630
Epoch 600/1000, Scaled MSE Loss: 0.9430
Epoch 700/1000, Scaled MSE Loss: 0.9210
Epoch 800/1000, Scaled MSE Loss: 0.9020
Epoch 900/1000, Scaled MSE Loss: 0.8910
Epoch 1000/1000, Scaled MSE Loss: 0.8836
```

8. Step 13 - Đổi ngược thang đo dự đoán (Inverse Scaling): Sau khi mô hình dự đoán ra y_hat_scaled trên tập test, việc đổi ngược về đơn vị giá ban đầu (Inverse Transformation) được thực hiện theo công thức: y_pred = y_hat_scaled * y_std + y_mean. Lưu ý: Trong hồi quy, không có ngưỡng quyết định 0.5 và không có ma trận nhầm lẫn, vì kết quả dự đoán là số thực liên tục.

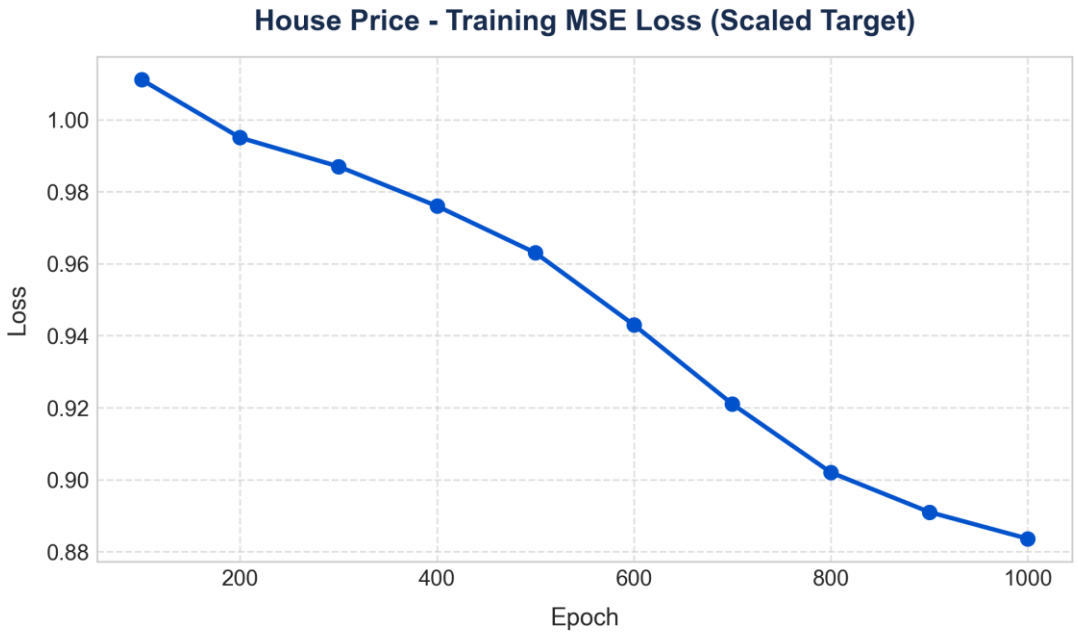
9. Step 14 - Đánh giá chỉ số sai số hồi quy: Tính toán các chỉ số sai số hồi quy trên đơn vị gốc: MAE = 31,585,039.76; RMSE = 1,502,158,095.88; MSE = 2.256×10^{18}; và R^2 = -2,351,754,048.62.

IV. So sánh với Random Forest và giải thích hiện tượng R^2 âm

Mô hình	MAE	MSE	RMSE	R^2
Random Forest - A02	6,303.08	1.059×10^9	32,544.55	0.1818
5-layer NN - A03	31,585,039.76	2.256×10^{18}	1,502,158,095.88	-2.35175×10^9

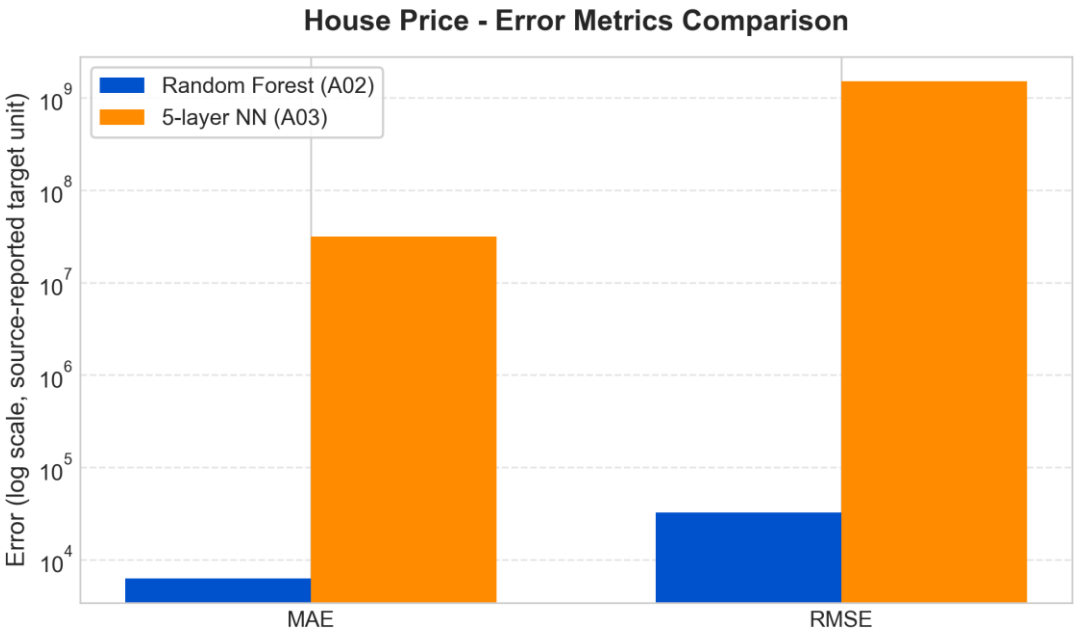
Phân tích nguyên nhân: Giải thích vì sao R^2 bị âm rất lớn (-2.35×10^9): Công thức của R^2 là: $R^2 = 1 - [\sum(y - \hat{y})^2 / \sum(y - \bar{y})^2]$. Trong đó mẫu số $\sum(y - \bar{y})^2$ là sai số khi chỉ dự đoán bằng giá trị trung bình thô $\bar{y}$ cho mọi mẫu. R^2 mang giá trị âm có nghĩa là dự đoán của mô hình còn kém hơn cả việc lấy giá trị trung bình đơn giản. Trong tập dữ liệu giá nhà, phân phối bị lệch phải rất mạnh (heavy-tailed) và chứa những bất động sản có giá trị khổng lồ (ngoại lai). Hàm mất mát MSE bình phương các khoảng cách sai số, khiến chỉ cần một vài điểm ngoại lai dự đoán chệch là tổng bình phương sai số bùng nổ lên tới 10^18. Trong khi Random Forest ở A02 sử dụng biến đổi log1p và cây quyết định phân vùng cục bộ nên không bị ảnh hưởng bởi ngoại lai, thì mạng nơ-ron sâu thô sơ với Full-batch GD đã bị các ngoại lai này chi phối hoàn toàn.

V. Trực quan hóa và phân tích đồ thị

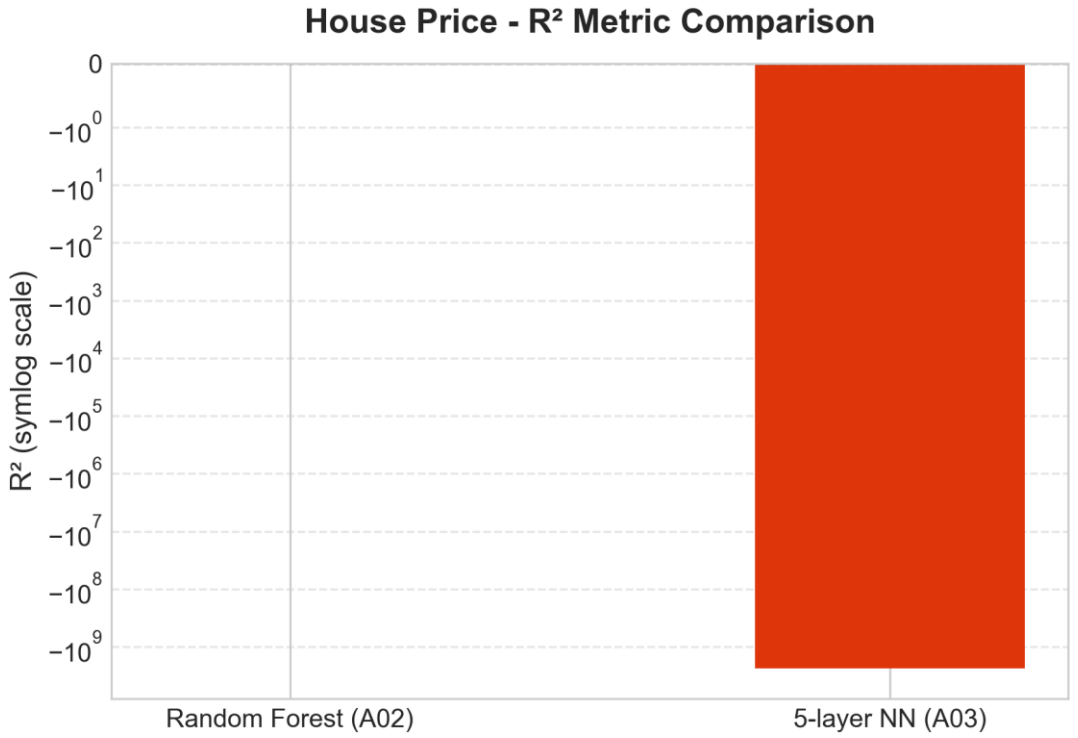


Hình 4: Đường cong hàm mất mát MSE trên dữ liệu chuẩn hóa qua 1000 epoch.

Đồ thị Hình 4 cho thấy trên tập train, loss chuẩn hóa giảm đều từ 1.0111 xuống 0.8836. Điều này khẳng định thuật toán tối ưu hoạt động đúng, nhưng sự suy giảm loss trên tập train không bảo đảm mô hình có khả năng khái quát hóa tốt trên tập test.



Hình 5: So sánh sai số MAE và RMSE giữa Random Forest và Mạng neural trên thang Log.



Hình 6: So sánh hệ số xác định R^2 trên trục đối xứng Symlog.

E. APPLICATION 3 - E-COMMERCE CUSTOMER BEHAVIOR

I. Mô hình và cấu trúc thực nghiệm

1. Mô tả bài toán: Notebook a03.ipynb sử dụng tập dữ liệu đánh giá sản phẩm thương mại điện tử (customer_behavior_clean.csv). Khác biệt quan trọng so với Assignment 02: Mô hình A03 không dùng cột văn bản nhận xét (Review Text) để xử lý ngôn ngữ tự nhiên, mà chỉ chọn 4 đặc trưng dạng bảng gồm: Age (Độ tuổi), Rating (Số sao đánh giá 1-5), Positive Feedback Count (Số lượt thích phản hồi) và Division Name (Tên bộ phận ngành hàng). Biến mục tiêu là Recommended IND ∈ {0, 1} (khách hàng có khuyến nghị sản phẩm hay không). Đây là bài toán phân loại nhị phân.

2. Mô hình Baseline cũ đối chiếu: Trong Assignment 02, bài toán cũ là dự đoán Department Name gồm 6 lớp phân loại, kết hợp 4 biến bảng với 8,000 từ khóa TF-IDF văn bản bằng mô hình Logistic Regression (Accuracy = 0.8296, F1 macro = 0.6339).

II. Kiến trúc mạng phân loại 5 lớp (5 -> 32 -> 16 -> 8 -> 4 -> 1)

Sau khi dùng dropna() và One-Hot cột Division Name (drop_first=True), tập dữ liệu có kích thước X = (23,472, 5) và y = (23,472, 1). Mạng neural 5 lớp có kiến trúc: 5 → 32 → 16 → 8 → 4 → 1, dùng ReLU ở 4 lớp ẩn và Sigmoid ở ngõ ra.

Lớp (Stage)	Kích thước biến đổi	Kích thước đầu ra	Hàm kích hoạt	Ý nghĩa / Vai trò
Input	Không có	N × 5	Không	5 đặc trưng dạng bảng (Age, Rating, Feedback, 2 dummy)
Layer 1	W1: 5×32, b1: 1×32	N × 32	ReLU	Học biểu diễn từ 5 đặc trưng dạng bảng (192 tham số)
Layer 2	W2: 32×16, b2: 1×16	N × 16	ReLU	Tổ hợp đặc trưng phi tuyến tầng 2 (528 tham số)
Layer 3	W3: 16×8, b3: 1×8	N × 8	ReLU	Nén không gian biểu diễn (136 tham số)
Layer 4	W4: 8×4, b4: 1×4	N × 4	ReLU	Biểu diễn 4 chiều trước quyết định (36 tham số)
Layer 5 (Output)	W5: 4×1, b5: 1×1	N × 1	Sigmoid	Tính điểm xác suất khách hàng khuyến nghị (5 tham số)

Tổng số tham số của mạng E-Commerce là: (5×32+32) + (32×16+16) + (16×8+8) + (8×4+4) + (4×1+1) = 192 + 528 + 136 + 36 + 5 = 897 tham số.

III. Chi tiết từng bước thực hiện (Step 1 đến Step 14)

1. Step 1 - Tiền xử lý dữ liệu: Sử dụng df[features + [target_col]].dropna() để loại bỏ các dòng có giá trị thiếu. Sau đó áp dụng pd.get_dummies(df, columns=['Division Name'], drop_first=True) để tạo các biến giả nhị phân cho cột Division Name. Tập đầu vào X có đúng 5 cột số: Age, Rating, Positive Feedback Count và 2 cột dummy của Division Name.

2. Step 2 & 3 - Chia Train/Test và Chuẩn hóa: Chia dữ liệu theo tỷ lệ 80% train (18,777 mẫu) và 20% test (4,695 mẫu) với seed 42 cố định. Chuẩn hóa đặc trưng X bằng mean và std của tập train để tránh rò rỉ dữ liệu.

3. Step 4 đến 12 - Huấn luyện mô hình: Khởi tạo mạng với phân phối He. Hàm forward() đưa dữ liệu qua 5 lớp và tính xác suất qua Sigmoid. Hàm mất mát là Binary Cross-Entropy. Lan truyền ngược backward() tính gradient bằng quy tắc chuỗi. Huấn luyện 1000 epoch với learning rate 0.01. Loss giảm rất mạnh từ 0.5505 (epoch 100) xuống 0.1621 (epoch 1000):

```
Epoch 100/1000, Loss: 0.5505
Epoch 200/1000, Loss: 0.4530
Epoch 300/1000, Loss: 0.3720
Epoch 400/1000, Loss: 0.2940
Epoch 500/1000, Loss: 0.2380
Epoch 600/1000, Loss: 0.2040
Epoch 700/1000, Loss: 0.1860
Epoch 800/1000, Loss: 0.1740
Epoch 900/1000, Loss: 0.1680
Epoch 1000/1000, Loss: 0.1621
```

4. Step 13 & 14 - Kết quả đánh giá: Trên tập test (4,695 mẫu), dự đoán với ngưỡng 0.5 đạt kết quả rất cao:

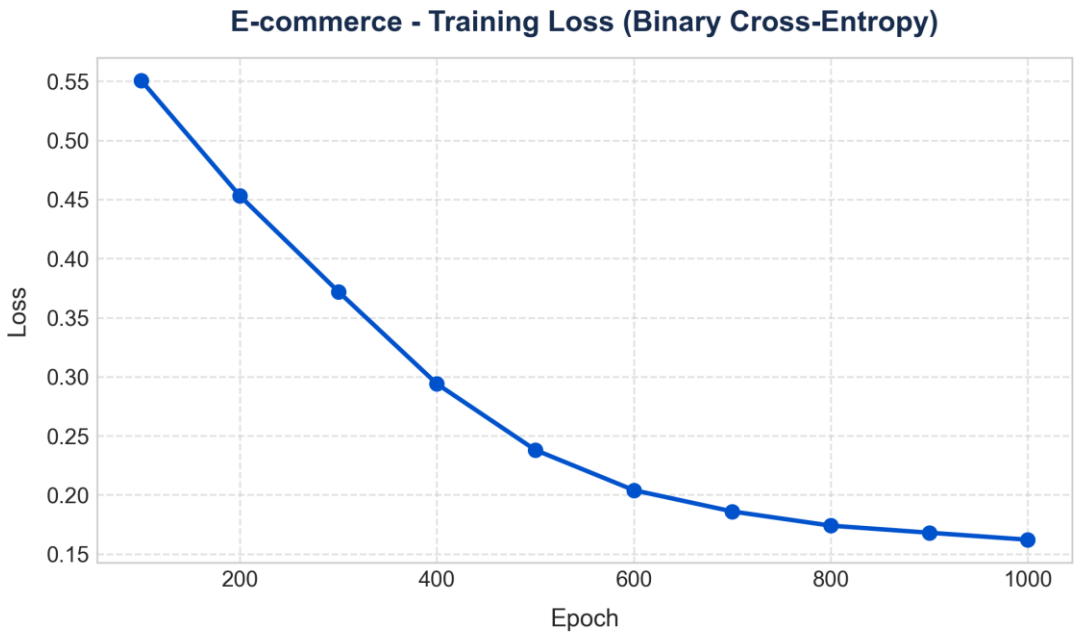
- **Độ chính xác:** Accuracy = 0.931416 (93.14%)
- **Độ chuẩn xác:** Precision = 0.963456 (96.35%)
- **Độ bao phủ:** Recall = 0.953008 (95.30%)
- **F1-Score:** F1-Score = 0.958204 (95.82%)
- **Ma trận nhầm lẫn:** TN = 682; FP = 140; FN = 182; TP = 3,691.

IV. So sánh và lưu ý phương pháp luận khi đối chiếu A02

Mô hình	Biến mục tiêu	Accuracy	Precision	Recall	F1-Score
Logistic Regression (A02)	Department Name (6 lớp)	0.8296	0.6178*	0.6612*	0.6339*
5-layer NN (A03)	Recommended IND (2 lớp)	0.9314	0.9635	0.9530	0.9582

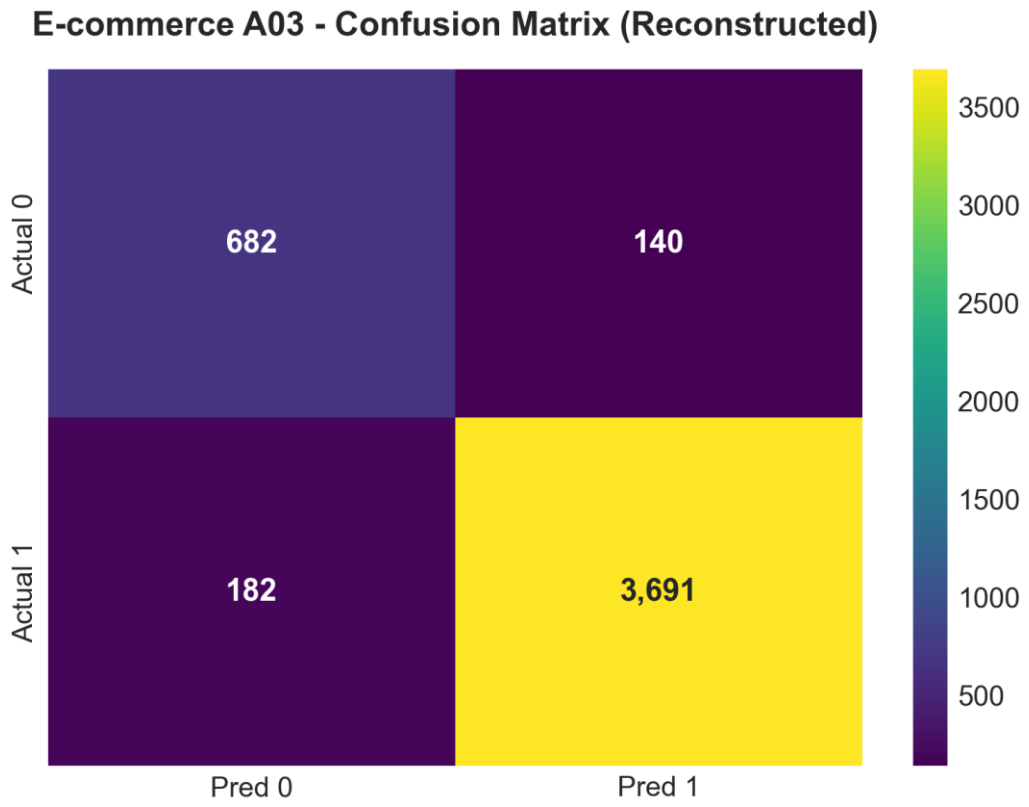
NGUYÊN TẮC SO SÁNH KHOA HỌC: Lưu ý rất quan trọng khi so sánh: Không được kết luận mô hình mạng nơ-ron A03 tốt hơn mô hình baseline A02 chỉ dựa trên các con số chỉ số ở trên. Lý do là hai bài toán có biến mục tiêu hoàn toàn khác nhau: A02 phân loại 6 lớp mất cân bằng dựa trên đặc trưng văn bản TF-IDF, trong khi A03 phân loại nhị phân trên 4 biến số dạng bảng. Cột Rating có tương quan đồng biến rất mạnh với việc khách hàng có khuyến nghị sản phẩm hay không (khách hàng đánh giá 4-5 sao gần như luôn khuyến nghị), điều này giải thích vì sao mô hình A03 đạt kết quả trên 93% rất dễ dàng. Muốn so sánh công bằng thì hai mô hình phải được huấn luyện trên cùng một biến mục tiêu và cùng cách chia dữ liệu.

V. Trực quan hóa và phân tích đồ thị

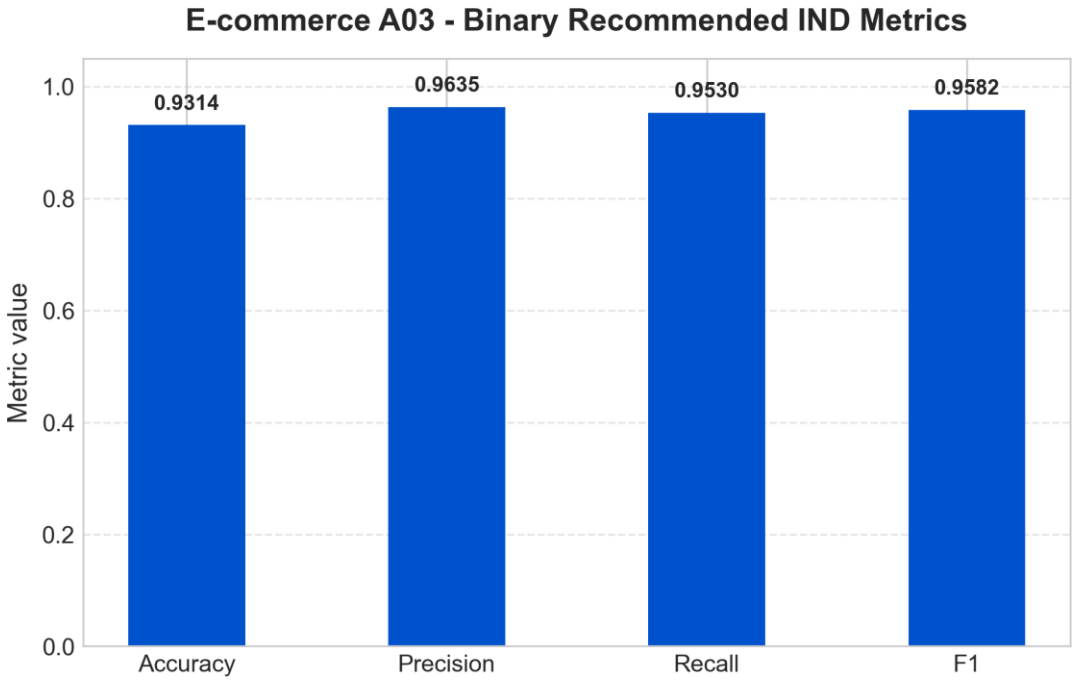


Hình 7: Đồ thị hàm mất mát Binary Cross-Entropy của bài toán E-Commerce.

Đồ thị Hình 7 cho thấy loss giảm rất dốc từ 0.5505 xuống 0.1621. Sự suy giảm nhanh chóng này phù hợp với kết quả độ chính xác cao trên tập test.



Hình 8: Ma trận nhầm lẫn tái dựng cho bài toán E-Commerce.



Hình 9: Biểu đồ 4 chỉ số đánh giá của mạng 5 lớp trên tập test.

F. TỔNG KẾT VÀ BÀI HỌC KINH NGHIỆM

I. Bảng tổng hợp đối sánh toàn diện 3 bài toán

Ứng dụng	Baseline A02	Deep Learning A03	Kết luận chính
Diabetes Prediction (22 đặc trưng, N=70,841)	SVM (RBF Kernel): • Accuracy: 0.7449 • F1-Score: 0.7779 • Recall: 0.8343	5-layer NN (BCE Loss): • Accuracy: 0.7144 • F1-Score: 0.7449 • Recall: 0.7866	Cùng biến mục tiêu. Mô hình SVM cũ tốt hơn mạng neural ở cả 4 chỉ số do tìm được biên phân cách tối ưu.
House Price Regression (276 đặc trưng, N=64,447)	Random Forest: • RMSE: 32,544.55 • R^2: 0.1818	5-layer NN (MSE Loss): • RMSE: 1.502×10^9 • R^2: -2.352×10^9	Baseline Random Forest vượt trội hoàn toàn; mô hình mạng neural bị ngoại lai chi phối nặng nề.
E-Commerce Behavior (5 đặc trưng, N=23,472)	Logistic Regression: • Target: Dept Name • F1-Macro: 0.6339	5-layer NN (BCE Loss): • Target: Recommended • F1-Score: 0.9582	Không so sánh trực tiếp vì bài toán và biến mục tiêu khác nhau (NLP 6 lớp vs Phân loại nhị phân).

II. So sánh giữa Machine Learning kinh điển và Deep Learning

Đánh giá tổng quan: Kết quả thực nghiệm trên 3 ứng dụng cho thấy rõ ràng: Mạng nơ-ron học sâu không tự động vượt trội hơn các mô hình Machine Learning kinh điển trên mọi tập dữ liệu. Trên dữ liệu dạng bảng (tabular data) có số chiều vừa phải như bài toán Diabetes và House Price, các mô hình SVM và Random Forest có ưu thế rất lớn vì chúng được thiết kế tiền xử lý kỹ lưỡng và ít bị ảnh hưởng bởi vấn đề tối ưu hóa cục bộ.

Đúng như slide bài giảng đã nêu: Việc tăng thêm số lớp chỉ làm tăng khả năng biểu diễn lý thuyết (representation capacity) của mô hình; hiệu quả thực tế đạt được còn phụ thuộc chặt chẽ vào đặc điểm phân phối của dữ liệu, hàm kích hoạt, hàm mất mát và thuật toán tối ưu hóa đi kèm.

III. Các bài học kỹ thuật cốt lõi

- **1. Tự cài đặt từ đầu (From Scratch):** Cả 3 notebook đều được tự cài đặt từ đầu bằng thư viện NumPy, không dùng PyTorch hay TensorFlow, giúp hiểu rõ bản chất từng phép nhân ma trận và từng bước tính đạo hàm.
- **2. Khởi tạo trọng số và hàm kích hoạt:** Việc dùng phân phối He (hệ số $\sqrt{(2/fan_in)}$) cho trọng số kết hợp với hàm kích hoạt ReLU ở 4 lớp ẩn là yếu tố quyết định giúp tín hiệu gradient không bị tắt dần khi truyền qua 5 lớp mạng.
- **3. Phối hợp hàm kích hoạt và hàm mất mát:** Bài toán phân loại nhị phân dùng hàm kích hoạt Sigmoid kết hợp với Binary Cross-Entropy giúp đạo hàm ngõ ra rút gọn thành $(\hat{y} - y)/N$. Bài toán hồi quy dùng hàm kích hoạt Linear kết hợp với Mean Squared Error giúp đạo hàm ngõ ra có dạng $2(\hat{y} - y)/N$.
- **4. Chuẩn hóa hai chiều trong bài toán hồi quy:** Trong bài toán hồi quy có giá trị lớn (như giá nhà), chuẩn hóa biến mục tiêu (Target Scaling) là thao tác bắt buộc để tránh hiện tượng nổ gradient. Tuy nhiên, hàm mất mát MSE rất nhạy cảm với ngoại lai cực đoan.
- **5. Đánh giá khách quan:** Đường cong loss suy giảm mượt mà trên tập train là điều kiện cần nhưng không đủ để khẳng định mô hình tốt; bắt buộc phải đánh giá trên tập test độc lập để kiểm tra khả năng tổng quát hóa.
- **6. Tính toàn vẹn thực nghiệm:** So sánh giữa hai thế hệ mô hình chỉ có ý nghĩa khoa học khi thực hiện trên cùng một biến mục tiêu, cùng không gian đặc trưng và cùng cách chia dữ liệu train/test.

IV. Định hướng nâng cấp mô hình

Để cải thiện hiệu năng của mạng nơ-ron tự xây dựng trên các bài toán thực tế phức tạp, các kỹ thuật sau đây có thể được bổ sung:

- • **Mini-Batch Gradient Descent:** Thay vì Full-Batch GD, việc chia nhỏ tập dữ liệu thành các lô kích thước 32 hoặc 64 mẫu giúp quá trình cập nhật tham số thường xuyên hơn và hỗ trợ vượt qua các điểm cực tiểu cục bộ kém.
- • **Thuật toán tối ưu nâng cao (Adam / Momentum):** Tích hợp thuật toán Adam hoặc RMSprop với tốc độ học thích nghi cho từng trọng số riêng biệt, giúp tăng tốc độ hội tụ và giảm dao động trong không gian tham số.
- • **Chính quy hóa (Regularization L2 / Dropout):** Thêm thành phần phạt trọng số L2 (Weight Decay) hoặc cơ chế Dropout ngẫu nhiên trong các lớp ẩn nhằm giảm hiện tượng quá khớp (overfitting) và tăng khả năng khái quát hóa.

V. Kết luận

Assignment 03 đã hoàn thành trọn vẹn mục tiêu kết nối giữa lý thuyết toán học của học sâu với mã nguồn lập trình thực tế. Quá trình thực nghiệm đã xây dựng đồ thị tính toán (computational graph), thực hiện lan truyền tiến, lưu trữ cache, tính toán đạo hàm bằng quy tắc chuỗi và cập nhật trọng số bằng Gradient Descent. Những kết quả thu được cả ở trường hợp thành công (như E-Commerce) lẫn trường hợp chưa đạt kỳ vọng (như House Price và Diabetes) đều mang lại những kinh nghiệm thực tế quý giá về cách hoạt động và giới hạn của mạng nơ-ron sâu.